

AMD Developer Hackathon: Participant Submission Guide

This document covers everything you need to build and submit a competitive entry. Exact evaluation inputs are intentionally omitted: your agent must be genuinely capable, not hardcoded to specific answers.

Track 1: General-Purpose AI Agent

What you are building

An AI agent that handles a wide variety of natural language tasks across multiple capability domains, using Fireworks AI models as efficiently as possible.

Why this task exists:

Enterprises want to control AI spend without sacrificing user experience: not every task needs a premium proprietary model. A common pattern is hosting a range of models in-house (open-source, fine-tuned, RAG-based) and only calling out to a premium API when genuinely necessary. Track 1 asks you to build that smart router: run as many local models as you need, they cost zero toward your score, and make as few external Fireworks API calls as possible while still clearing the accuracy gate.

Capability categories

Your agent will be evaluated across all eight categories. Build for all of them:

#	Category	What it covers
1	Factual knowledge	Explaining concepts, definitions, and how things work
2	Mathematical reasoning	Multi-step arithmetic, percentages, word problems, projections
3	Sentiment classification	Labelling sentiment and justifying the classification

#	Category	What it covers
4	Text summarisation	Condensing passages to a specific format or length constraint
5	Named entity recognition	Extracting and labelling entities (person, org, location, date)
6	Code debugging	Identifying bugs in code snippets and providing corrected implementations
7	Logical / deductive reasoning	Constraint-based puzzles where all conditions must be satisfied
8	Code generation	Writing correct, well-structured functions from a spec

What to submit

A **Docker image** pushed to a public registry (e.g. GitHub Container Registry, Docker Hub).
Check out the Image Architecture requirement at the bottom of this document

Your container must:

1. Read tasks from `/input/tasks.json` on startup

```
JSON
[
  { "task_id": "t1", "prompt": "Summarise the following text in one sentence: ..." },
  { "task_id": "t2", "prompt": "..." }
]
```

2. Write results to `/output/results.json` before exiting

```
JSON
[
  { "task_id": "t1", "answer": "..." },
  { "task_id": "t2", "answer": "..." }
]
```

```
{ "task_id": "t2", "answer": "..." }  
]
```

Practice tasks (not the real evaluation set)

These are illustrative examples only — not the real grading tasks (those stay hidden). Use them to validate your container's input/output handling locally before using a real submission slot.

JSON

```
[  
  { "task_id": "practice-01", "prompt": "What is the capital of Australia, and  
    what body of water is it near?" },  
  { "task_id": "practice-02", "prompt": "A store has 240 items. It sells 15% on  
    Monday and 60 more on Tuesday. How many items remain?" },  
  { "task_id": "practice-03", "prompt": "Classify the sentiment of this review:  
    The battery life is great, but the screen scratches too easily." },  
  { "task_id": "practice-04", "prompt": "Summarize the following in exactly one  
    sentence: [your own sample paragraph here]." },  
  { "task_id": "practice-05", "prompt": "Extract all named entities and their  
    types from: Maria Sanchez joined Fireworks AI in Berlin last March." },  
  { "task_id": "practice-06", "prompt": "This function should return the max of a  
    list but has a bug: def get_max(nums): return nums[0]. Find and fix it." },  
  { "task_id": "practice-07", "prompt": "Three friends, Sam, Jo, and Lee, each own  
    a different pet: cat, dog, bird. Sam does not own the bird. Jo owns the dog. Who  
    owns the cat?" },  
  { "task_id": "practice-08", "prompt": "Write a Python function that returns the  
    second-largest number in a list, handling duplicates correctly." }  
]
```

Environment variables:

The harness injects these at runtime. Read them from the environment: do not hardcode values or bundle a .env file in your image.

```
Python  
import os
```

```

api_key    = os.environ["FIREWORKS_API_KEY"]    # provided by harness – do not use
your own
base_url   = os.environ["FIREWORKS_BASE_URL"]   # route ALL Fireworks calls through
this URL
models     = os.environ["ALLOWED_MODELS"].split(",") # exact model IDs published
on launch day

```

For local development you can use a .env file, but your submitted container must read these purely from the environment: the harness will inject the real values at evaluation time.

Variable	Description
FIREWORKS_API_KEY	Provided by the harness — use this key, not your own
FIREWORKS_BASE_URL	Base URL for all Fireworks API calls — must be used to configure your client
ALLOWED_MODELS	Comma-separated list of permitted Fireworks AI model IDs, published on launch day

Important: All API calls must go through `FIREWORKS_BASE_URL`. Calls that bypass this URL will not be recorded and the submission will score zero tokens. Do not hardcode model IDs: read from `ALLOWED_MODELS` at runtime.

Rules

- Exit code `0` on success, non-zero on failure
- Maximum runtime: **10 minutes**
- Only models in `ALLOWED_MODELS` are permitted, calls to other models invalidate the submission
- `/output/results.json` must be valid JSON, malformed output scores zero
- **Local models and tokens used locally count as zero** for the final score; all Fireworks API calls must go through `FIREWORKS_BASE_URL`; local model inference inside the container is permitted and counts toward accuracy, but not toward the token score.
- Do not hardcode or cache answers; evaluation uses unseen prompt variants
- Image compressed size must not exceed 10GB — larger images are rejected before pulling
- Submissions are rate-limited to 10 per hour per team

- Grading environment: **4 GB RAM, 2 vCPU**. If bundling a local model, size it to fit within these limits (2B–3B 4-bit quantized models are safe; 7B 4-bit fills the full RAM budget, leaving no room for agent code).

Scoring

1. **Accuracy gate**: LLM-Judge evaluates each answer against the expected intent. Submissions below the accuracy threshold are excluded from the leaderboard.
2. **Token efficiency**: submissions that pass the accuracy gate are ranked ascending by total tokens recorded by the judging proxy. Fewer tokens = higher rank.

A note on token counting: The underlying task prompts are identical for every team, but your own system prompt (verbosity instructions, formatting requests, etc.) affects your input token count, and your model's response length affects output tokens. Don't over-optimize output length early, focus first on your routing logic and which local models you use. Output-length tuning is a good later-stage optimization once your router is solid.

Troubleshooting: why did my submission fail?

If your submission doesn't score as expected, here's what each status means and how to fix it. Most of these also apply to Track 2.

Status	What it means & how to fix it
PULL_ERROR	We couldn't pull your Docker image. Confirm it's public, and includes a linux/amd64 manifest (Apple Silicon builds need <code>docker buildx build --platform linux/amd64</code>).
RUNTIME_ERROR	Your container ran but exited with a non-zero error code. Check your own container logs locally — something in your agent code crashed.
TIMEOUT	Your container didn't finish within the 10-minute limit. Check for hangs, infinite loops, or excessive retries in your agent.
OUTPUT_MISSING	Your container exited cleanly but never wrote <code>/output/results.json</code> . Confirm your code writes this file before exiting.
INVALID_RESULTS_SCHEMA	<code>/output/results.json</code> isn't in the right format. Each entry must be a JSON object with both

	a task_id and an answer field.
MODEL_VIOLATION	You called a Fireworks model that isn't in the published ALLOWED_MODELS list. Only call models from that list, read it from the env var at runtime, don't hardcode it.
IMAGE_TOO_LARGE	Your image is over the 10 GB compressed size limit. Trim unnecessary layers/dependencies from your Docker image.
ACCURACY_GATE_FAILED	Your container ran fine, but your answers scored below the accuracy threshold. This is a quality issue with your agent's answers, not an infrastructure problem.

Note: you may also see a flagged: **ZERO_API_CALLS** marker alongside your result; this is not a failure. It just means your submission made zero calls through the Fireworks proxy (e.g. a local-model-only agent), which is a valid strategy per the local models rule above.

Track 2: Video Captioning Agent

What you are building

An AI agent that watches a video clip and generates a caption in a requested style. Build your pipeline to handle a wide variety of video content: different scenes, settings, subjects, and visual styles.

Styles your agent must support

Style	Description
formal	Professional, objective, factual tone
sarcastic	Dry, ironic, lightly mocking
humorous_tech	Funny, with technology or programming references
humorous_non_tech	Funny, everyday humour with no technical jargon

What to submit

A **Docker image** pushed to a public registry.

Your container must:

1. Read tasks from `/input/tasks.json` on startup

JSON

```
[
  {
    "task_id": "v1",
    "video_url": "https://storage.example.com/clips/clip1.mp4",
    "styles": ["formal", "sarcastic", "humorous_tech", "humorous_non_tech"]
  }
]
```

2. Write results to `/output/results.json` before exiting

JSON

```
[
  {
    "task_id": "v1",
    "captions": {
      "formal": "...",
      "sarcastic": "...",
      "humorous_tech": "...",
      "humorous_non_tech": "..."
    }
  }
]
```

No inference log is required for Track 2.

Check out the Image Architecture requirement at the bottom of this document

Environment variables

No API key or model restriction is injected. You may call any model, API, or framework: use your own credentials inside the container.

Example clips

Use these to develop and test your agent. Evaluation uses a hidden set your agent has never seen, ensure your pipeline generalises beyond these examples.

Clip	URL	Content
v1	https://storage.googleapis.com/amd-hackathon-clips/1860079-uhd_2560_1440_25fps.mp4	Urban autumn boulevard with golden trees and city traffic
v2	https://storage.googleapis.com/amd-hackathon-clips/13825391-uhd_3840_2160_30fps.mp4	Orange kitten among green foliage in a garden
v3	https://storage.googleapis.com/amd-hackathon-clips/3044693-uhd_3840_2160_24fps.mp4	Office worker at a desktop computer in a modern open-plan office

Rules

- Exit code 0 on success, non-zero on failure
- Maximum runtime: **10 minutes**
- No model restriction: any model, framework, or API is permitted
- **Video clips in the hidden evaluation set are 30 seconds to 2 minutes in length**
- Output must include a caption for every requested style for every clip, missing styles score zero for that clip
- `/output/results.json` must be valid JSON, malformed output scores zero
- Image compressed size must not exceed 10GB — larger images are rejected before pulling
- Submissions are rate-limited to 10 per hour per team

Scoring

Each caption is scored on two dimensions by an LLM-Judge:

1. **Caption accuracy** (0–1): how faithfully the caption reflects the video content
2. **Style match** (0–1): how well the caption matches the requested tone

Final score = weighted average across all clips and all four styles. Evaluation uses a hidden set of ~12 clips spanning varied content (nature, urban, animals, people, sports, food, weather, technology). Agents that only work on the three example clips above will score poorly.

Track 3: Unicorn (Open Innovation)

What you are building

An original AI application that uses AMD compute resources. There is no fixed task, we are looking for the most innovative, technically impressive, and practically useful projects.

What to submit

Item	Required
GitHub repository URL	Yes
Demo video	Yes
Slide deck	Yes
Live demo / hosted URL	Optional but recommended

Note: automated pre-screening only inspects the GitHub repository, slide deck (PDF), and live demo/hosted URL — it does not process the demo video.

No Docker image is required for Track 3.

Judging

Submissions are pre-screened automatically for AMD resource usage and originality, then reviewed by human judges. AMD compute usage is a requirement: projects that do not demonstrate it will be disqualified.

General rules (all tracks)

- Your container must start and be ready within **60 seconds**
- Response time per request must be under **30 seconds**
- All responses must be in **English**
- Do not hardcode or cache answers to specific inputs — evaluation uses unseen variants

- Container images must be publicly pullable at submission time

Image architecture requirement

The judging VM runs *linux/amd64*. Your image must include a *linux/amd64* manifest or it will fail to pull and score zero.

If you build on Apple Silicon (M1/M2/M3), add *--platform linux/amd64* to your build command:

Shell

```
docker buildx build --platform linux/amd64 --tag your-image:latest --push .
```

Standard *linux/amd64* builds (e.g. built on Intel/AMD or GitHub Actions) are fine without any changes.